Cockpit Data Resilience Plan

Unbreakable School Data Sync Architecture

Version: 1.0

Date: October 13, 2026

Author: Fireball (Managing Director) with expert team input

Status: Draft for Meheldi's review

Executive Summary

Cockpit depends on 6 school data sources. If any one fails, parents lose visibility into their child's education. This plan designs a **three-tier fallback architecture** that ensures data keeps flowing even when individual components fail.

Core principle: No single point of failure. Every data source has 3 transport methods. Every data type has 2+ overlapping sources. Every infrastructure layer has a backup.

Target SLAs:

- Grade data: < 4 hours stale during school days
- Attendance: < 24 hours stale
- Initial onboarding dump: < 15 minutes from credential entry
- System availability: 99.5% (excluding school-side outages)

---

1. Data Source Inventory

1.1 ParentVUE (Parent Portal)

- **URL:** parentvue.cobbk12.org
- **Auth:** Native username/password (parent's own)
- **Data:** Grades, assignments, attendance, class schedule, teacher info
- **Primary transport:** Mobile REST API (`/api/v1/mobile/PXPWebServices/AttemptLogin` → Bearer token → `GetStudentData`)

- **Fallback transport 1:** SOAP API (`/Service/PXPCommunication.asmx`, `ProcessWebServiceRequest`)
- **Fallback transport 2:** Web scrape (`PXP2_*.aspx` pages)
- **Critical note:** Cobb logs every failed login. NEVER retry bad credentials.

1.2 CTLS Parent (Parent Portal)

- **URL:** ctlsparent.cobbk12.org
- **Auth:** Form-based login (parent's credentials)
- **Data:** Announcements, assignments, supplementary grade data
- **Primary transport:** Form POST + session cookies + `/api/v2` endpoints
- **Fallback transport:** Web scrape with session persistence

1.3 StudentVUE (Student Portal)

- **URL:** studentvue.cobbk12.org
- **Auth:** Microsoft SSO (student's school email + password via Office 365)
- **Data:** Grades, assignments (student view, often fresher than ParentVUE)
- **Primary transport:** Browser/SSO automation (laptop workers)
- **Note:** Mobile REST API does NOT accept SSO credentials. Must use browser path.

1.4 CTLS Learn (Student LMS)

- **URL:** Cobb County CTLS Learn platform
- **Auth:** Microsoft SSO (same as StudentVUE)
- **Data:** Course materials, announcements, upcoming assignments, syllabus
- **Primary transport:** Browser/SSO automation (laptop workers)

1.5 Office 365 Email (Student)

- **URL:** outlook.office365.com
- **Auth:** Microsoft SSO
- **Data:** Teacher emails, school announcements, assignment notifications
- **Primary transport:** Microsoft Graph API (with proper OAuth)
- **Fallback transport:** IMAP (if enabled by district)

1.6 Office 365 Calendar (Student)

- **URL:** outlook.office365.com/calendar
- **Auth:** Microsoft SSO
- **Data:** School events, assignment due dates, class schedule

- **Primary transport:** Microsoft Graph API
- **Fallback transport:** CalDAV / ICS export

---

2. Three-Tier Fallback Architecture

Tier 1: Transport Fallback (Same Source, Different Method)

For each source, if the primary transport fails, automatically try the next:

	Primary	Fallback 1	Fallback 2
	REST API	SOAP API	Web scrape
t	API v2	Form + scrape	Manual
E	Browser SSO	N/A	Manual
	Browser SSO	N/A	Manual
	Graph API	IMAP	Manual
dar	Graph API	ICS export	Manual

Rules:

- Transport failures (timeout, 500, connection reset) → try next transport immediately
- Auth failures (401, "invalid credentials") → STOP, do not try other transports, alert user
- Each transport attempt logged with timestamp and result

Tier 2: Source Fallback (Different Source, Overlapping Data)

When a source is completely unavailable, use overlapping data from another source:

	Primary Source	Fallback Source	Last Resort
	ParentVUE	StudentVUE (laptop)	Manual entry
s	ParentVUE	CTLS Learn	Teacher email parse
	ParentVUE	StudentVUE	Manual
ents	CTLS Parent	O365 Email	Manual
dule	ParentVUE	CTLS Learn	Manual

Conflict resolution: When two sources disagree on a grade:

- StudentVUE wins for current grades (student view is often fresher)

- ParentVUE wins for official/reporting grades
- If difference > 1.0 points, flag for manual review
- Always show which source each data point came from

Tier 3: Infrastructure Fallback (Different Compute Location)

	Primary	Fallback	Last Resort
	Cloudflare Worker	N/A	—
	Hatch VM	N/A	—
	MacBook launchd	Windows Task Scheduler	Manual trigger
	User enters data	—	—

Key insight: Laptops are on the home network and CAN reach school APIs when the VM/cloud cannot. They are the most reliable infrastructure layer.

---

3. Component Design

3.1 Cloud Sync Orchestrator (NEW — to be built)

Purpose: Central coordinator that decides what to sync, when, and via which path.

Responsibilities:

- Maintain sync schedule per crew member per source
- Track last successful sync per source per transport
- Detect failures and trigger fallback transports/sources
- Queue credential validation requests
- Emit health metrics

Location: Cloudflare Worker (cron trigger) + Supabase for state

Database tables:

- `sync_schedule` — what to sync when
- `sync_attempts` — log of every attempt (transport, result, duration)
- `sync_health` — current health per source per crew member

3.2 Laptop Sync Workers (EXISTS — being deployed)

Purpose: Reliable data pulls from the home network.

Current state:

- MacBook: `~/parentvue-sync/` with launchd scheduling (hourly, school days)
- Windows: `~/parentvue-sync/` with Task Scheduler (hourly, school days)
- Credentials in OS keychain (never in files)

Enhancements needed:

- Add StudentVUE/CTLS Learn browser automation
- Add O365 Graph API pulls
- Report health back to cloud orchestrator
- Handle credential expiry gracefully

3.3 Credential Vault

Current:

- `crew_credential_pending` table (encrypted with AES-256-GCM via Web Crypto API)
- Encryption: `v1:` prefix + base64(iv[12] || ciphertext), random 12-byte IV per encryption
- Key from `CALENDAR_TOKEN_KEY` env var (64 hex chars = 32 bytes), never committed
- OS keychain on laptops (macOS `security` CLI, Windows Credential Manager)
- Secure Vault for user-provided credentials (write-only for agents)

Security properties verified:

- AES-256-GCM with random IVs (no IV reuse)
- Key never in code or logs
- Passwords live in memory only during request, never logged
- Auth failures never retried (Cobb lockout protection)

Enhancements needed:

- Automatic expiry detection (proactive re-auth prompts)
- Credential rotation without data loss
- Audit log of credential access
- Key rotation procedure (re-encrypt all rows with new key)

3.4 Health Monitor (EXISTS — needs enhancement)

...

Enhancements needed:

- Per-source health (not just aggregate)

- Per-transport health (is REST down but SOAP up?)
- Auto-remediation (trigger fallback when primary fails)
- Alert escalation (warning → alert → page)

---

4. New User Onboarding Pipeline

Goal: Credentials in → first data dump in < 15 minutes.

Flow:

User adds child → enters ParentVUE credentials

Immediate: Validate credentials via best available transport (2 min timeout)

If valid: Queue initial full sync (all sources, all data types)

Cloud orchestrator picks up queue within 1 minute

Laptop worker (if available) OR **cloud worker** pulls initial dump

Progressive UI: Show "Syncing..." → classes appear → grades fill in → complete

If no laptop worker: Cloud worker does best-effort pull; prompt user to install laptop worker for reliability

Fallback: If initial sync fails, show clear error with retry button. Never leave user with blank dashboard.

---

5. Failure Mode Analysis

5.1 School API Changes

- **Likelihood:** Medium (Cobb updates 1-2x/year)
- **Impact:** High (all transports for that source may break)
- **Mitigation:** Transport fallback (if REST breaks, SOAP may still work). Web scrape as last resort. Monitor for API version headers.
- **Detection:** Health monitor alerts when all transports fail for a source.

5.2 Credential Expiry / Lockout

- **Likelihood:** Medium (passwords expire, users change them)
- **Impact:** High (sync stops completely for that source)
- **Mitigation:** Proactive expiry detection (warn 7 days before). NEVER retry bad credentials (lockout risk). In-app re-auth flow.

- **Detection:** Auth failure on sync → immediate user notification.

5.3 Network / Egress Blocks

- **Likelihood:** Medium (seen twice in Oct 2026)
- **Impact:** Medium (VM/cloud can't reach school APIs, but laptops can)
- **Mitigation:** Infrastructure fallback to laptops. Laptops are on home network, not affected by VM egress issues.
- **Detection:** Health monitor tracks per-infrastructure-layer success rates.

5.4 School SSO Changes (Microsoft)

- **Likelihood:** Low (Microsoft is stable)
- **Impact:** High (all student sources break simultaneously)
- **Mitigation:** Browser automation is resilient to SSO flow changes. Monitor Microsoft login page for changes.
- **Detection:** All student sources fail simultaneously → likely SSO issue.

5.5 Data Format Changes

- **Likelihood:** Medium (gradebook schema evolves)
- **Impact:** Medium (parser breaks, data looks wrong)
- **Mitigation:** Schema validation on every pull. If schema changes, log warning and use best-effort parsing. Alert for manual review.
- **Detection:** Parser extracts 0 classes when it previously extracted 6 → schema change.

---

6. Testing Plan

6.1 Transport Fallback Tests

For each source, simulate primary transport failure and verify fallback engages:

- [ ] ParentVUE: Block REST API → verify SOAP engages
- [ ] ParentVUE: Block REST+SOAP → verify web scrape engages
- [ ] CTLS Parent: Block API v2 → verify form+scrape engages

6.2 Source Fallback Tests

- [ ] Disable ParentVUE → verify StudentVUE data fills gaps

- [ ] Disable all APIs → verify UI shows stale data with timestamps (not blank)

6.3 Infrastructure Fallback Tests

- [ ] Block VM egress → verify laptop workers continue syncing
- [ ] Take down cloud orchestrator → verify laptops continue on schedule

6.4 Credential Tests

- [ ] Enter wrong password → verify NO retry, immediate user notification
- [ ] Expire credential → verify proactive warning 7 days before
- [ ] Change password → verify re-auth flow works end-to-end

6.5 Onboarding Tests

- [ ] New user adds child → verify first data dump in < 15 min
- [ ] New user with no laptop → verify cloud-only path works
- [ ] Onboarding during school API outage → verify graceful error with retry

6.6 Chaos Tests

- [ ] Kill Supabase mid-sync → verify graceful degradation
- [ ] Corrupt gradebook data → verify parser handles gracefully
- [ ] Simulate Cobb API version change → verify detection and alert

---

7. Loopholes and Edge Cases

7.1 Multiple Children

- ParentVUE `GetChildListData` returns multiple children. Must correctly map each child to their crew member.
- **Fix:** Store `child_int_id` per crew member. Validate on every sync.

7.2 Custody / Access Changes

- If a parent loses access to a child's ParentVUE, the API returns empty (not error).
- **Fix:** Detect "0 children" as potential access loss, not just "no data." Alert user.

7.3 Summer / Break Periods

- During summer, gradebook is empty. This is normal, not a failure.
- **Fix:** School calendar awareness. Don't alert on empty data during known breaks.

7.4 Grade Disputes Between Sources

- ParentVUE shows 85, StudentVUE shows 87. Which is right?
- **Fix:** Show both with source labels. Use conflict resolution rules (Section 2, Tier 2).

7.5 Partial Sync Failures

- Grades sync OK, but attendance fails. Is the sync "successful"?
- **Fix:** Per-section success tracking. Overall status = "partial" if any section fails. Retry failed sections independently.

7.6 Clock Skew

- Laptop clock wrong → sync timestamps wrong → health monitor false alerts.
- **Fix:** Use server timestamps (Supabase `now()`), not client timestamps.

---

8. Implementation Roadmap

Phase 1: Foundation (Week 1-2)

- [ ] Build cloud sync orchestrator (Worker + Supabase tables)
- [ ] Enhance health monitor (per-source, per-transport tracking)
- [ ] Fix ParentVUE validation from Workers (or document laptop-only validation)

Phase 2: Fallback Automation (Week 3-4)

- [ ] Implement automatic transport fallback in orchestrator
- [ ] Implement source fallback with conflict resolution
- [ ] Add laptop worker health reporting to cloud

Phase 3: Onboarding (Week 5-6)

- [ ] Build new user onboarding pipeline (< 15 min to first data)
- [ ] Progressive UI (syncing → partial → complete states)
- [ ] Laptop worker installer (one-click setup)

Phase 4: Hardening (Week 7-8)

- [ ] Chaos testing (all scenarios in Section 6)
- [ ] Credential lifecycle automation (expiry detection, rotation)
- [ ] Documentation and runbooks

---

9. Monitoring and Alerting

9.1 Dashboards

- **Sync Health Dashboard:** Per crew member, per source, per transport. Green/yellow/red.
- **Onboarding Funnel:** Credentials entered → validated → first sync → complete. Drop-off analysis.
- **Infrastructure Health:** Cloud Worker, VM, MacBook, Windows — last seen, success rate.

9.2 Alert Thresholds

Condition	Severity	Action
No sync in 4h (school day)	Warning	Log, check next run
No sync in 12h (school day)	Alert	Notify user, try fallback
No sync in 24h	Critical	Page on-call, manual intervention
Auth failure	Alert	Immediate user notification
All transports down for source	Critical	Page on-call

9.3 Runbooks

For each alert, a step-by-step runbook:

- How to diagnose
- How to manually trigger fallback
- When to escalate to Meheldi
- When to contact school IT

---

10. Expert Team Findings

10.1 Distributed Systems Architecture Review

...

Critical design principles:

Laptop-first for school pulls — Laptops are on the home network, pass Conditional Access, and hold SSO cookie jars. Cloudflare Workers are a thin tier; the VM never pulls.

Every source has 3 transport tiers with automatic step-down/step-up.

Source fallback is explicit per domain — grade cells filled from StudentVUE when ParentVUE is down, flagged `via_fallback`.

Credentials are never retried on auth failure — one bad attempt → `cred-suspect` → stop → alert.

Partial success is success — a sync pulling 4 of 6 sources lands those 4.

Dashboard never shows blank — stale data carries freshness badges; missing data carries recovery CTAs.

New tables proposed:

- `sync_health` — per crew_member × source × transport health tracking
- `credential_events` — append-only auth lifecycle audit (no secrets)

Transport step-down policy:

- Step down T1→T2 after 3 consecutive transport failures in 24h (not auth failures)
- Step up after 48h green + 3 consecutive shadow-mode successes
- Every step change logged to `sync_health`

...

10.2 Python/Backend Code Review

Scope: 6 files reviewed end-to-end (parentvue_pull.py, studentvue_api.py, parentsquare_pull.py, http.py, run_parent_pull.py, parentvue-laptop-sync.py) plus adjacent modules (config.py, health.py, sink.py, pookie_health.py).

CRITICAL FINDINGS

C1. Empty/garbage API responses count as success — silent data erasure

A failure chain turns "no data" into "successful sync with no data":

`StudentVueAPI.call()` passes `expect_status=None`; when server returns HTTP 200 with non-JSON (WAF block page), returns `(200, None)` instead of raising.

`normalize_gradebook()` degrades any unknown shape to `{"classes": []}` — never raises.

`pull_all()` records non-raising sections as present.

`ok = any(v is not None...)` → True.

`zero_grade_guard` only iterates current grades — empty vector produces zero flags.

Staging overwrites good snapshot with empty one → pushes empty grades to Supabase.

A single WAF-blocked response silently erases the grade record while logging success.

...

C2. Raw password in SOAP XML — special characters break login silently

...

...

HIGH FINDINGS

H1. Auth-failure by substring causes extra bad-credential logins (lockout risk)

...

...

H2. Laptop worker has no local staging — push failure = data loss

...

Fix: Write payload to local staging dir before pushing; on failure, leave spooled and retry next run. Gate push: if 0 classes while last good had classes, abort without pushing.

H3. Orchestrator re-reads staging file instead of using in-hand result (race condition)

...

...

H4. Transient network error nukes valid ParentSquare session

...

Fix: Distinguish transport failure (keep jar) from invalid session (re-login).

...

...

...

H6. Missing --env-file hard-fails even with env vars set

...

...

MEDIUM FINDINGS (M1-M11)

- M1: No `Retry-After` honoring on 429/503 — burns all retries too fast
- M2: ParentSquare mid-run session expiry kills remaining sections

- M3: 401 token-refresh can cause ~6 extra AttemptLogins per run
- M4: Laptop push uses weaker retry (no jitter, no Retry-After)
- M5: Stuck "syncing" heartbeat — crash leaves permanent stale status
- M6: Laptop exits 0 on degraded runs (schedulers can't distinguish)
- M7: Cookie jar saved without restrictive permissions (0644)
- M8: Non-atomic state.json/latest.json writes (crash = corrupt JSON)
- M9: Silent telemetry-package version drift on laptops
- M10: ParentSquare retry set inconsistent with http.py
- M11: SOAP login probe doubles gradebook traffic

Single Points of Failure

One hostname per portal — no DNS fallback

One credential store per platform — missing entry = hard fail

ParentVUE transport ladder is REST→SOAP only in-module (web scrape lives elsewhere)

ParentSquare has no real fallback (API + scrape share same session)

Laptop worker: single Supabase URL, single RPC — push is most fragile link

Suggested fix order: C1 → C2 → H1 → H2 → H5 → H3 → H4/H6 → M-batch.

C1+H5 together are the "silent data erasure" path — treat as one incident-class fix.

10.3 Security Review

...

Credential storage:

- Parent credentials: OS keychain only (macOS Keychain, Windows Credential Manager DPAPI)
- Student SSO: No password stored — persisted browser cookie jar on trusted laptop
- Graph refresh tokens: OS keychain, rotated on each use
- Server side: Supabase never sees passwords/tokens

Critical rules:

- Passwords live in memory only during request, never logged
- Auth failures never retried (lockout protection)
- `credential_events` table must never store secrets (audit only)

Appendix A: API Reference

ParentVUE Mobile REST API

- **Base:** `https://parentvue.cobbk12.org/api/v1/mobile/PXPWebServices`
- **Auth:** `POST /AttemptLogin` with `Authorization: Basic base64(user:pass)`
- **Body:** `{"arguments": {"request": "{\"userID\": null}"}}`
- **Headers:** `Content-Type: application/json`, `Accept: application/json`
- **Response:** Bearer token in JSON envelope
- **Data:** `POST /GetStudentData` with token + `childIntID`

ParentVUE SOAP API

- **URL:** `https://parentvue.cobbk12.org/Service/PXPCommunication.asmx`
- **Method:** `ProcessWebServiceRequest`
- **Auth:** Username/password in SOAP envelope
- **Methods:** `Gradebook`, `Attendance`, `StudentInfo`

Microsoft Graph API (O365)

- **Auth:** OAuth 2.0 with school tenant
- **Email:** `GET /me/messages`
- **Calendar:** `GET /me/calendar/events`

Appendix B: Database Schema

crew_grade_syncs

...

- `'student-laptop'` — primary student-side pull
- `'parentvue-laptop'` — backup parent-side pull from laptop
- `'parentvue-cloud'` — cloud worker pull (future)
- `'manual'` — user-entered data

crew_credential_pending

Encrypted credential queue. Fields:

- `crew_member_id`, `pilot_id`
- `credential_type` ('parentvue', 'student', 'ctls-parent')
- `username_enc`, `password_enc` (AES-256-GCM)

- `processed_at` (null = pending)

sync_health (proposed)

Per-source health tracking:

- `crew_member_id`, `source`, `transport`
- `last_success_at`, `last_attempt_at`, `consecutive_failures`
- `status` ('healthy', 'degraded', 'down')

End of Plan